

helper_perception 패키지 정리

목적: 실물 RPLIDAR C1 입력을 받아 LiDAR scan을 필터링하고, 전방/후방 장애물 여부를 Bool 토픽으로 발행하는 perception 계층 구성

1. 현재 구현 범위

- RPLIDAR C1 드라이버 실행 launch 구성
- raw scan 토픽 remap 구성
- LaserScan 필터링 노드 구현
- 장애물 판단 노드 구현
- front/rear 공용 재사용 구조로 확장
- front 단독, rear 단독, dual LiDAR 통합 launch 구성

2. 토픽 구조

구분	raw scan	filtered scan	obstacle result
Front	/perception/scan/front/raw	/perception/scan/front	/perception/obstacle/front
Rear	/perception/scan/rear/raw	/perception/scan/rear	/perception/obstacle/rear

센서 원본 데이터에는 /raw 를 붙이고, 필터링 후 데이터는 /raw 없이 사용한다.

3. 실행 명령어

3-1. 빌드

```
cd /home/jiming/workspace/helper_robotics
colcon build --symlink-install
source install/setup.bash
```

3-2. Front LiDAR 전체 실행

```
ros2 launch helper_perception front_lidar_perception.launch.py
```

3-3. Rear LiDAR 전체 실행

```
ros2 launch helper_perception rear_lidar_perception.launch.py
```

3-4. Front + Rear 동시 실행

```
ros2 launch helper_perception dual_lidar_perception.launch.py \
  front_serial_port:=/dev/lidar_front \
  rear_serial_port:=/dev/lidar_rear
```

3-5. 토픽 확인

```
ros2 topic echo /perception/scan/front/raw
ros2 topic echo /perception/scan/front
ros2 topic echo /perception/obstacle/front
ros2 topic echo /perception/scan/rear/raw
ros2 topic echo /perception/scan/rear
ros2 topic echo /perception/obstacle/rear
```

3-6. 주파수 확인

```
ros2 topic hz /perception/scan/front/raw
ros2 topic hz /perception/scan/rear/raw
```

4. 주요 파일 역할

파일	역할
scan_filter_node.py	LaserScan에서 지정 각도 범위와 거리 범위만 남기고, 나머지 값은 <code>inf</code> 로 변환
obstacle_detector_node.py	감지 각도 범위 내 최소 거리를 계산하고 threshold 이하이면 장애물로 판단
front_lidar_c1.launch.py	Front C1 드라이버 실행 및 <code>scan -> /perception/scan/front/raw</code> remap
rear_lidar_c1.launch.py	Rear C1 드라이버 실행 및 <code>scan -> /perception/scan/rear/raw</code> remap
perception_lidar.launch.py	필터 노드와 장애물 판단 노드를 공용 인스턴스로 실행
front_lidar_perception.launch.py	Front 드라이버 + front perception 통합 실행
rear_lidar_perception.launch.py	Rear 드라이버 + rear perception 통합 실행
dual_lidar_perception.launch.py	Front + rear 전체 파이프라인 동시 실행

5. 코드 동작 요약

5-1. scan_filter_node

- 입력 LaserScan 메시지의 header와 메타데이터는 유지

- 각 포인트의 각도를 계산하여 설정한 각도 범위 안인지 확인
- NaN, inf, 최소 거리 미만, 최대 거리 초과 값은 무효 처리
- 무효 값과 사용하지 않는 각도 범위는 inf 로 변환

5-2. obstacle_detector_node

- 필터링된 scan에서 감지 각도 범위 안의 유효 거리만 사용
- 그 중 최소 거리를 계산
- 최소 거리가 threshold 이하이면 true, 아니면 false 발행
- 디버그 로그는 약 1초 간격으로 출력

6. 주요 파라미터

파라미터	기본값	의미
angle_min_deg	-90.0	필터링에 사용할 최소 각도
angle_max_deg	90.0	필터링에 사용할 최대 각도
min_valid_range	0.15	이 값보다 가까운 측정값은 무효 처리
max_valid_range	8.0	이 값보다 먼 측정값은 무효 처리
detection_angle_min_deg	-30.0	장애물 판단 최소 각도
detection_angle_max_deg	30.0	장애물 판단 최대 각도
obstacle_distance_threshold	0.5	이 거리 이하이면 장애물로 판단

파라미터 변경 예시

```
ros2 launch helper_perception perception_lidar.launch.py \
  obstacle_distance_threshold:=0.7 \
  detection_angle_min_deg:=-20.0 \
  detection_angle_max_deg:=20.0
```

7. 실물 검증 절차

1. LiDAR raw scan 수신 확인
2. RViz에서 raw scan과 filtered scan 비교
3. 정면 장애물 배치 후 obstacle 토픽이 true 로 바뀌는지 확인
4. 장애물 제거 후 false 복귀 확인
5. 감지 범위 바깥 장애물에 대해서는 false가 유지되는지 확인

RViz 확인 포인트

- Fixed Frame 은 현재 scan frame과 맞춰야 함
- 현재 front 기본 frame은 front_laser, rear 기본 frame은 rear_laser
- 최종 시스템에서는 base_link -> front_laser, base_link -> rear_laser TF 정의가 필요

8. 추후 수정 및 보완 필요 사항

- 실차 장착 후 front/rear 감지 각도 범위 재조정
- 실차 주행 환경 기준 거리 threshold 재조정
- LiDAR 장착 위치 반영을 위한 URDF/TF 정리
- Jetson 환경에서 안정적인 포트 식별을 위해 udev rule 적용 권장
- rear LiDAR 실물 연결 후 실제 장착 방향 기준으로 파라미터 개별 튜닝
- 장애물 판단 결과를 behavior 계층과 연결
- 필요 시 정지/감속 등 다단계 상태로 확장

9. 현재 확인 완료 항목

- 실물 RPLIDAR C1 연결 확인
- Front raw scan remap 확인
- 필터링 scan 발행 확인
- 정면 장애물 판단 로직 동작 확인
- RViz에서 LiDAR scan 표시 확인
- front/rear 공용 실행 구조 구성 완료

10. 다음 개발 단계 제안

1. helper_bringup 패키지 분리 여부 결정
2. URDF에 front/rear LiDAR frame 반영
3. rear LiDAR 실물 검증
4. behavior node에서 /perception/obstacle/front, /perception/obstacle/rear 구독
5. 최종적으로 제어 계층과 연결